POLITECNICO
MILANO 1863

Software Engineering

Definitions
The process and the product
Phases of the development process

M Camilli, E Di Nitto, M Rossi

Why software engineering is important

POLITECNICO MILANO 1863

- Software is everywhere and our society is now totally dependent on software-intensive systems
- Society could not function without software
  - Healthcare Systems
  - Transportation Systems
  - Energy Systems
  - Manufacturing Systems
  - …
- Software failures cannot be tolerated… but they still happen!

Example: worldwide IT outage on July 19, 2024

- CrowdStrike crashed enterprise Windows-based systems worldwide (approximately 8.5 million devices)
  - Airlines including American Airlines, Delta, KLM, Lufthansa, Ryanair, SAS, and United
  - Airports (e.g., Gatwick, Luton), healthcare, and media organizations (e.g., MTV, Sky, BBC)
  - Financial organizations including the London Stock Exchange, Lloyds Bank, and Visa
  - Sporting bodies including F1 teams (e.g., Aston Martin, Mercedes, and Williams) and the Paris 2024 Olympic Organizing Committee
- Array out of bounds read issue!
  - https://www.computerweekly.com/feature/CrowdStrike-update-chaos-explained-What-you-need-to-know
  - https://www.crowdstrike.com/wp-content/uploads/2024/08/Channel-File-291-Incident-Root-Cause-Analysis-08.06.2024.pdf

How a CrowdStrike Update Knocked the World Offline

Behind the simple Microsoft software update that caused a worldwide IT breakdown

ET EXPLAINER

Aashish Aryan

New Delhi: Two days after an update from CrowdStrike triggered an error in Microsoft Windows operating systems and affected 8.5 million Windows-run devices across the globe, both companies have released an analysis of the sequence of events of July 19, why the crash happened and how such crashes can be avoided in the future. In an update to its users late on Saturday, CrowdStrike said that a sensor configuration update to Windows systems was released early on July 19. This update caused a logic error in Windows-based systems, which in turn resulted in these de-

BIG MELTDOWN

• **A sensor** update on CrowdStrike's Falcon Sensor trig-

Software engineering: definition

POLITECNICO
MILANO 1863

- Field of computer science dealing with software systems
  - large and complex
  - built by teams
  - exist in many versions
  - last many years
  - undergo changes
- Multi-person construction of multi-version software

Software engineer: required skills

POLITECNICO MILANO 1863

- **Programming skills not enough!**
  - Programmer:
    - develops a complete program
    - works on known specifications
    - works individually
  - Software engineer:
    - identifies requirements and develops specifications
    - designs a component to be combined with other components, developed, maintained, used by others; component can become part of several systems
    - collaborates with others to ensure components show adequate qualities
    - works in a team

Skills of software engineers

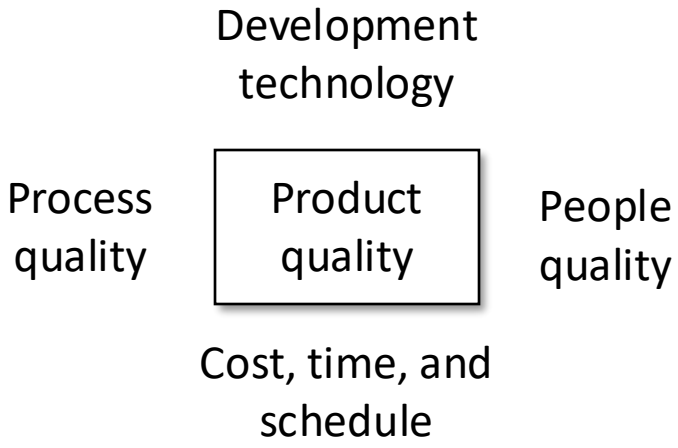
POLITECNICO MILANO 1863

- Technical
- Project management
- Cognitive
- Enterprise organization
- Interaction with different cultures
- Domain knowledge

- The quality of human resources is of primary importance

The software product

- Different from traditional types of products
  - intangible
    - difficult to describe and evaluate
  - malleable
  - human intensive
    - does not involve any trivial manufacturing process
- Aspects affecting product quality



Software product qualities – ISO/IEC 25010:2023 Software Quality Model

POLITECNICO MILANO 1863

Software Product Quality

Functional suitability
- Functional completeness
- Functional correctness
- Functional appropriateness

Performance efficiency
- Time behavior
- Resource utilization
- Capacity

Reliability
- Faultlessness
- Availability
- Fault tolerance
- Recoverability

Security
- Confidentiality
- Integrity
- Non-repudiation
- Accountability
- Authenticity
- Resistance

Compatibility
- Co-existence
- Interoperability

Safety
- Ability to set operational constraints
- Ability to identify risks
- Ability to fail safe
- Ability to warn about hazards
- Ability to safe integration

Mantainability
- Modularity
- Reusability
- Analyzability
- Modifiability
- Testability

Flexibility
- Adaptability
- Scalability
- Installability
- Replaceability

Interaction capability
- Appropriateness recognizability
- Learnability
- Operability
- User error protection
- User engagement
- Inclusivity
- User assistance
- Self-descriptiveness

Process qualities: productivity

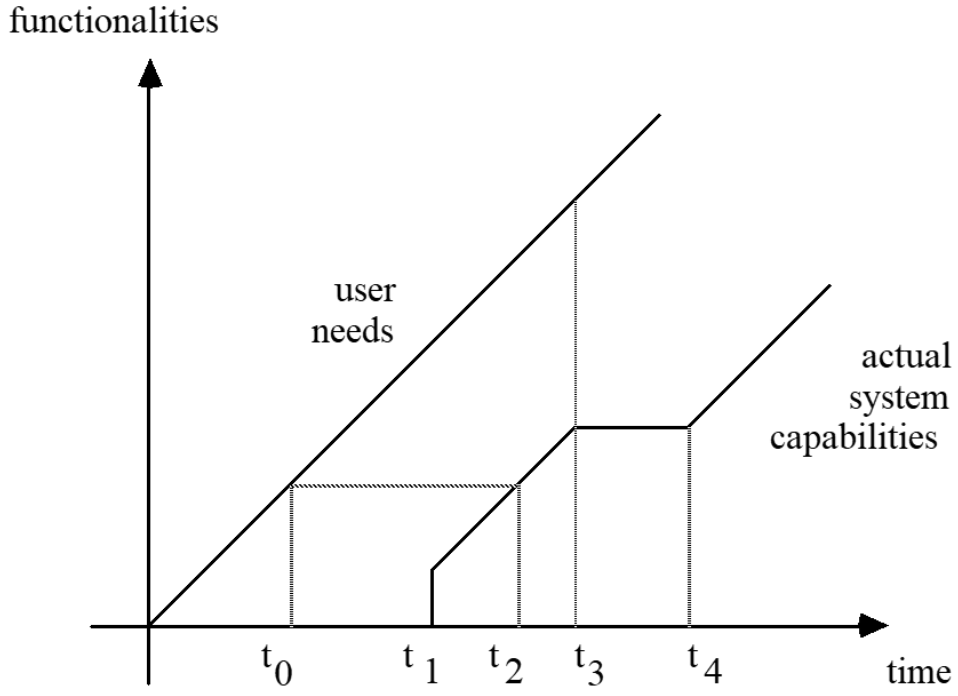
- Ability to produce a “good” amount of product
- How can we measure it? → Delivered items by unit of effort

lines of code (and variations)
function points

person month
WARNING: persons and months cannot be interchanged

Process qualities: timeliness

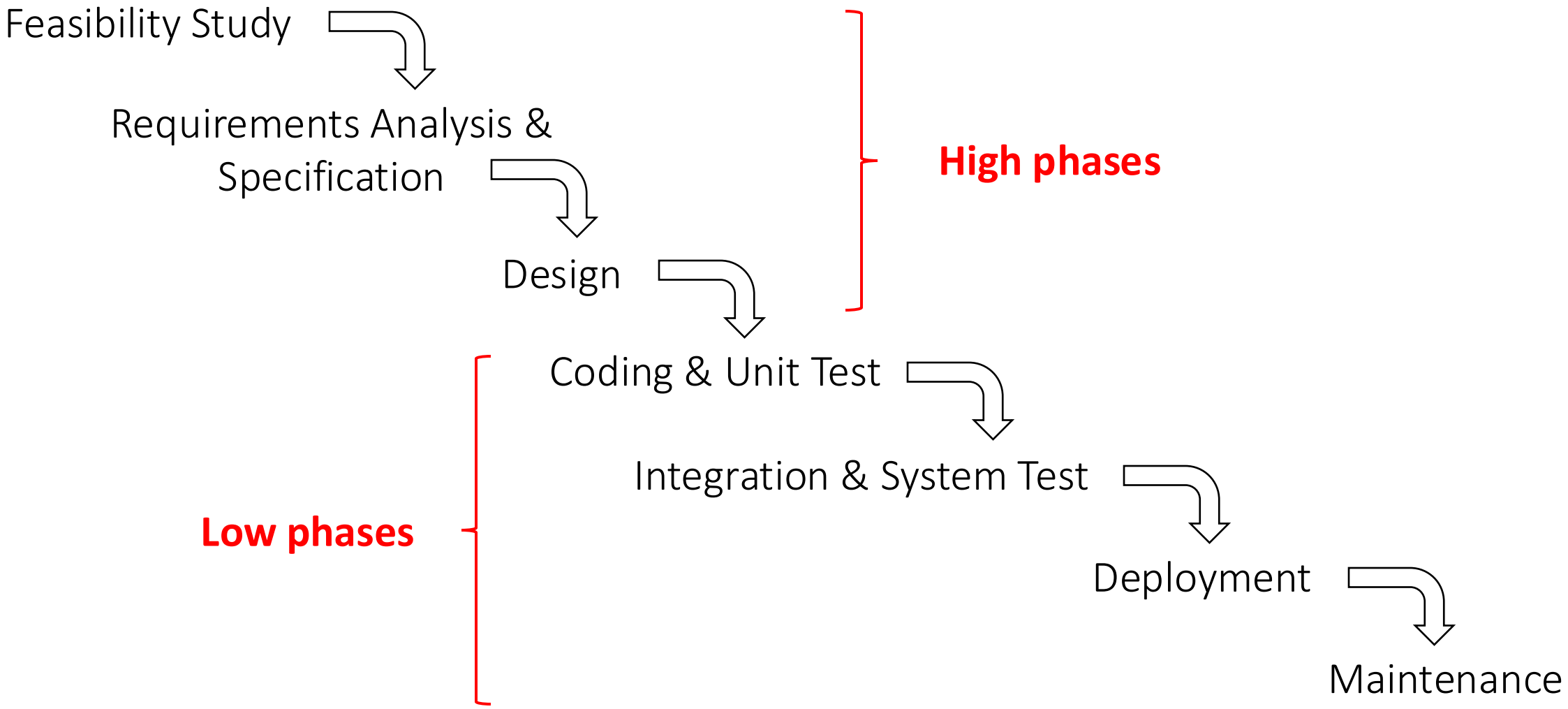
- Ability to respond to change requests in a timely fashion



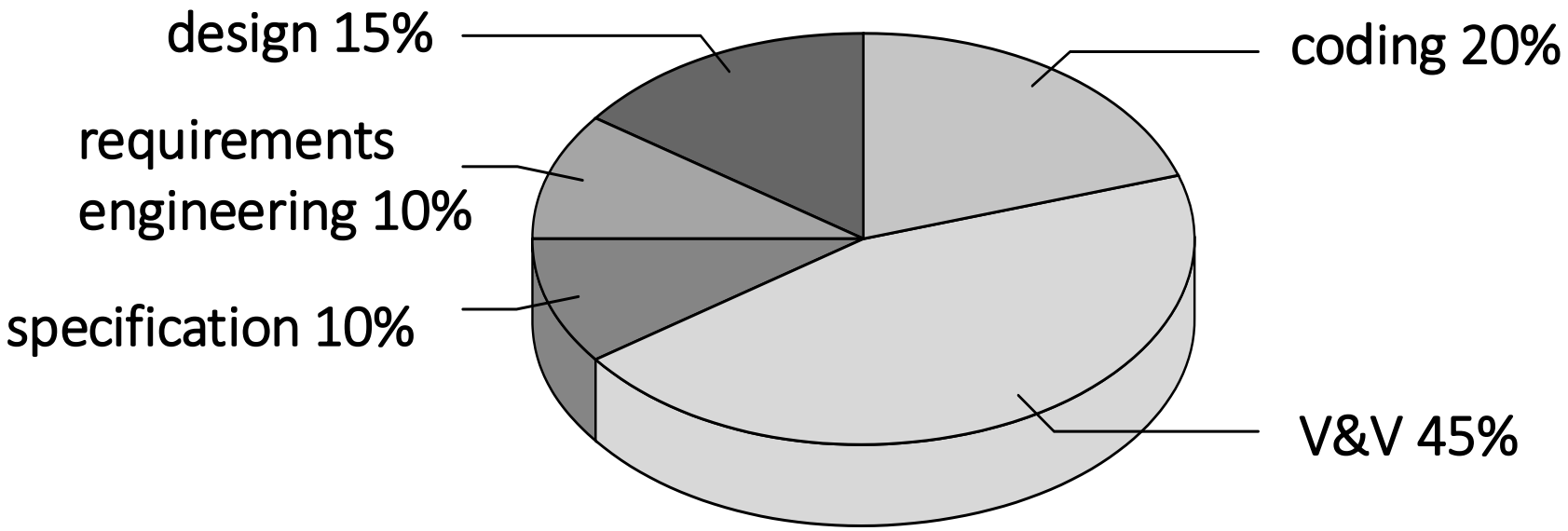
Software lifecycles

- Initially, no reference model: Just code & fix
- As a reaction to the many problems: traditional “waterfall” model
  - identify phases and activities
  - force linear progression from a phase to the next
  - no returns (they are harmful)
    - better planning and control
  - standardize outputs (artifacts) from each phase
  - Software like manufacturing
- Then, flexible processes: iterative models, agile methods, DevOps

A waterfall organization



Effort distribution (van Vliet 2008)



Maintenance

- Corrective maintenance: deals with the repair of faults or defects found ≈ 20%
- Evolution
  - **adaptive** maintenance: consists of adapting software to changes in the environment (the hardware or the operating system, business rules, government policies…) ≈ 20%
  - **perfective** maintenance: mainly deals with accommodating to new or changed user requirements ≈ 50%
  - **preventive** maintenance: concerns activities aimed at increasing the system's maintainability ≈ 10%

Correction vs evolution

- Distinction can be unclear, because specifications are often incomplete and ambiguous
- This causes problems because specs are often part of a contract between developer and customer
  - early frozen specs can be problematic, because they are more likely to be wrong

Problems of software evolution

POLITECNICO
MILANO 1863

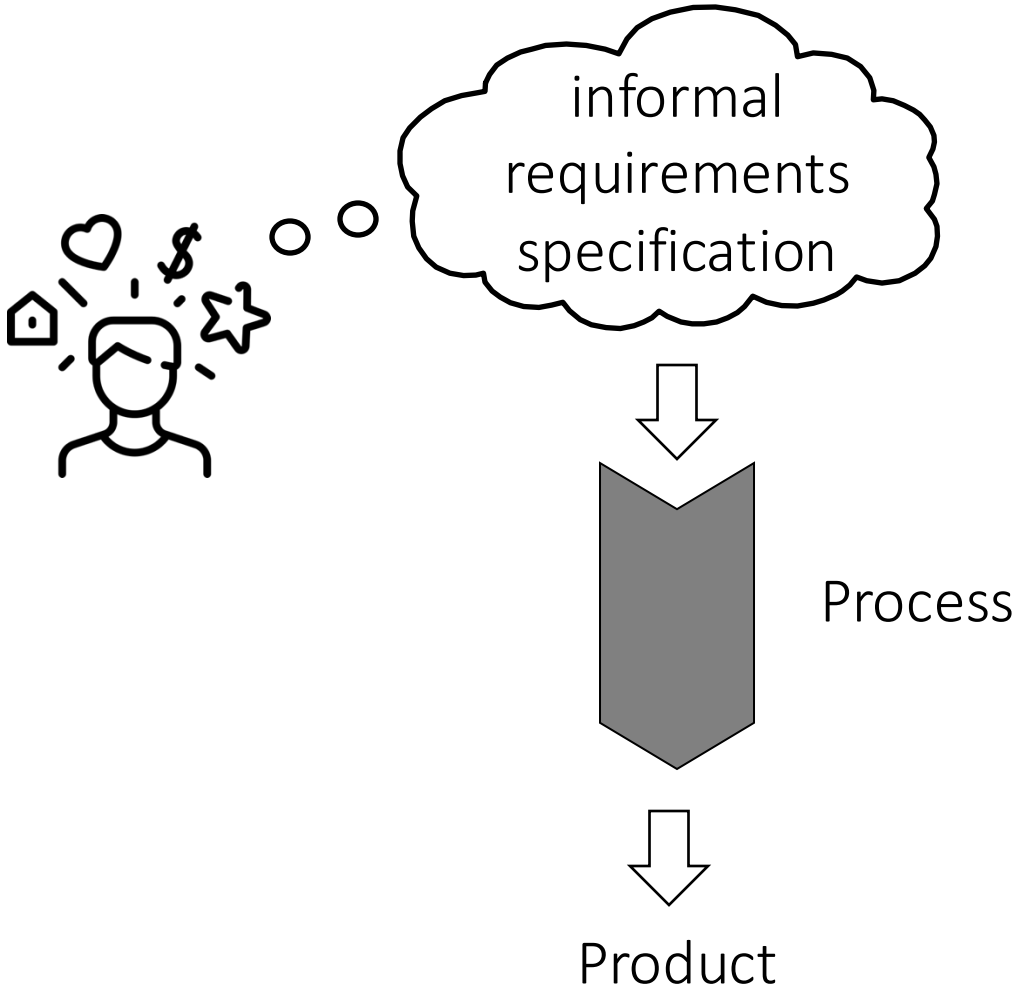
- It is (almost) never anticipated and planned; this causes disasters
- Software is very easy to change
  - often, under emergency, changes are applied directly to code
  - inconsistent state of project documents

How to face evolution

- Good engineering practice:
  - first modify design, then change implementation
  - apply changes consistently in all documents
- Likely changes must be anticipated
- Software must be designed to accommodate changes in a cost-effective way

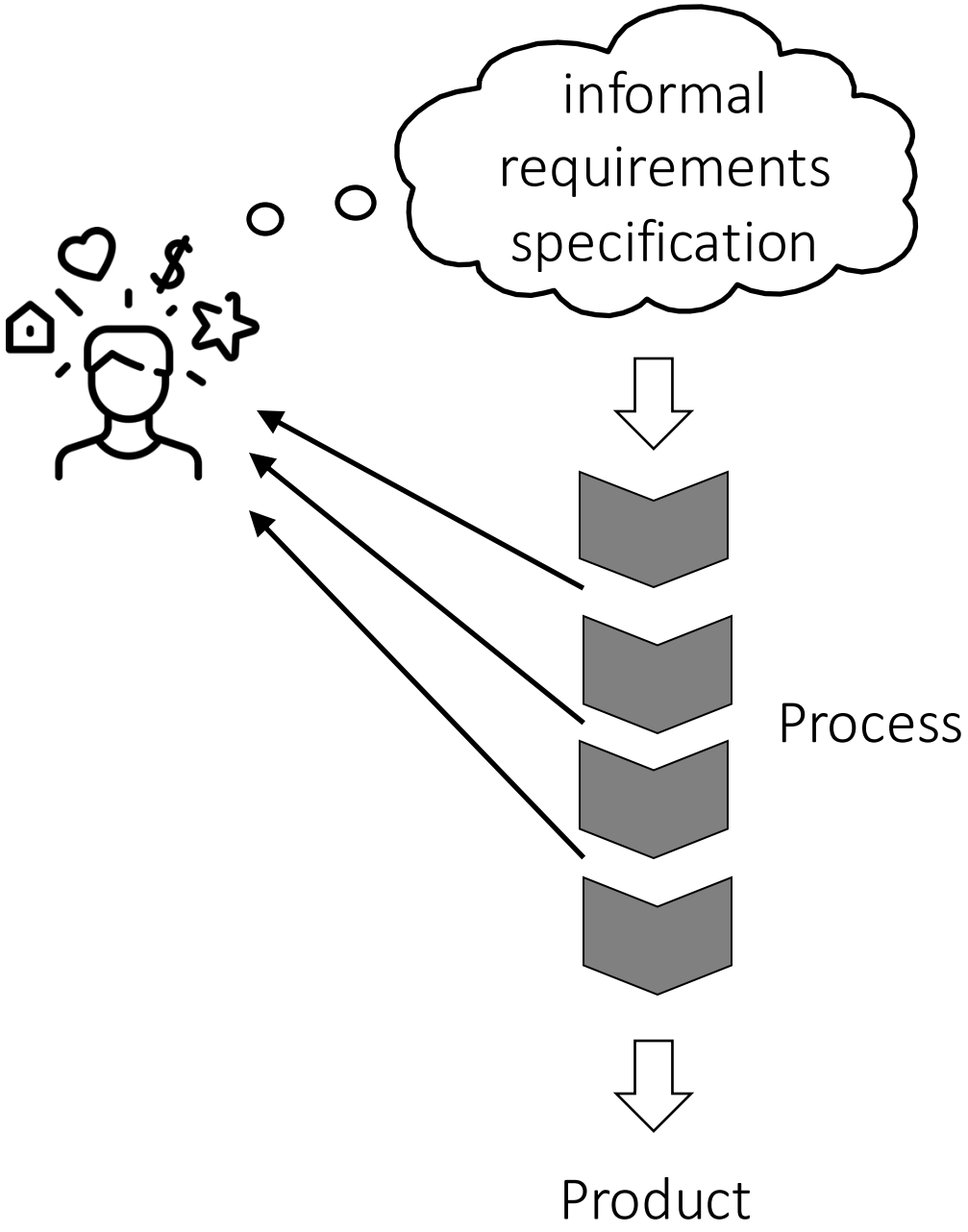
- **This is one of the main goals of software engineering**

Waterfall is “black box”

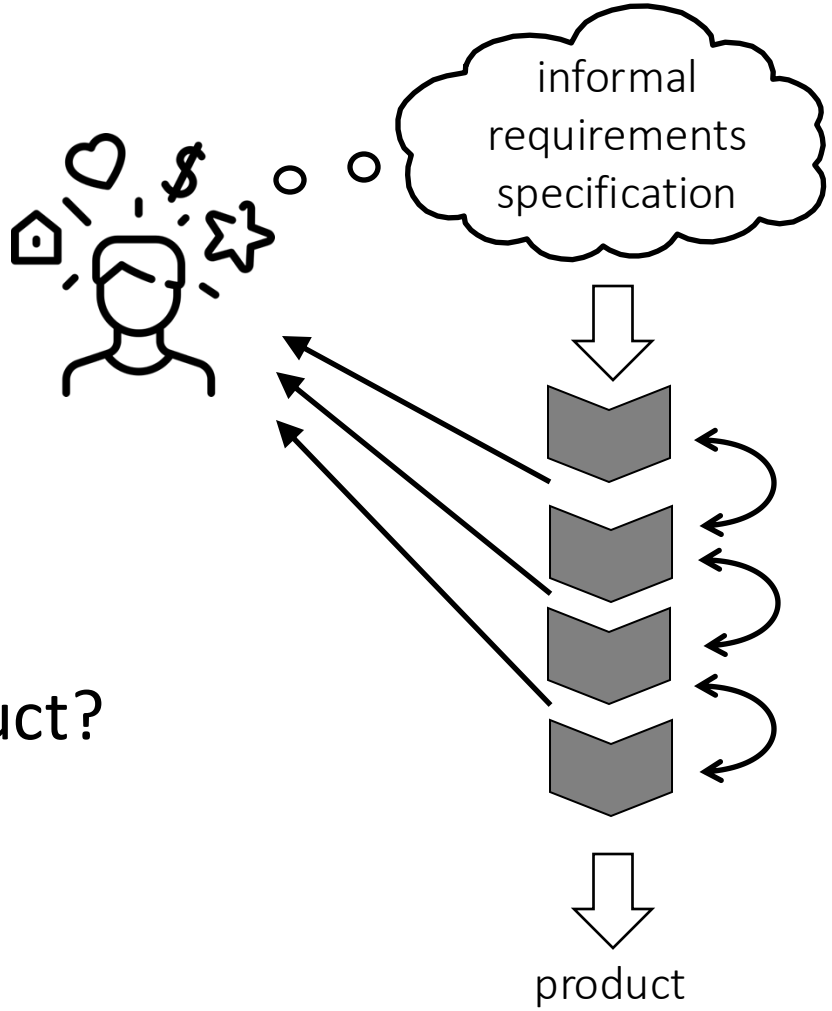


Need for transparency

- Transparency allows early check and change via feedback
- It supports flexibility



Verification and validation



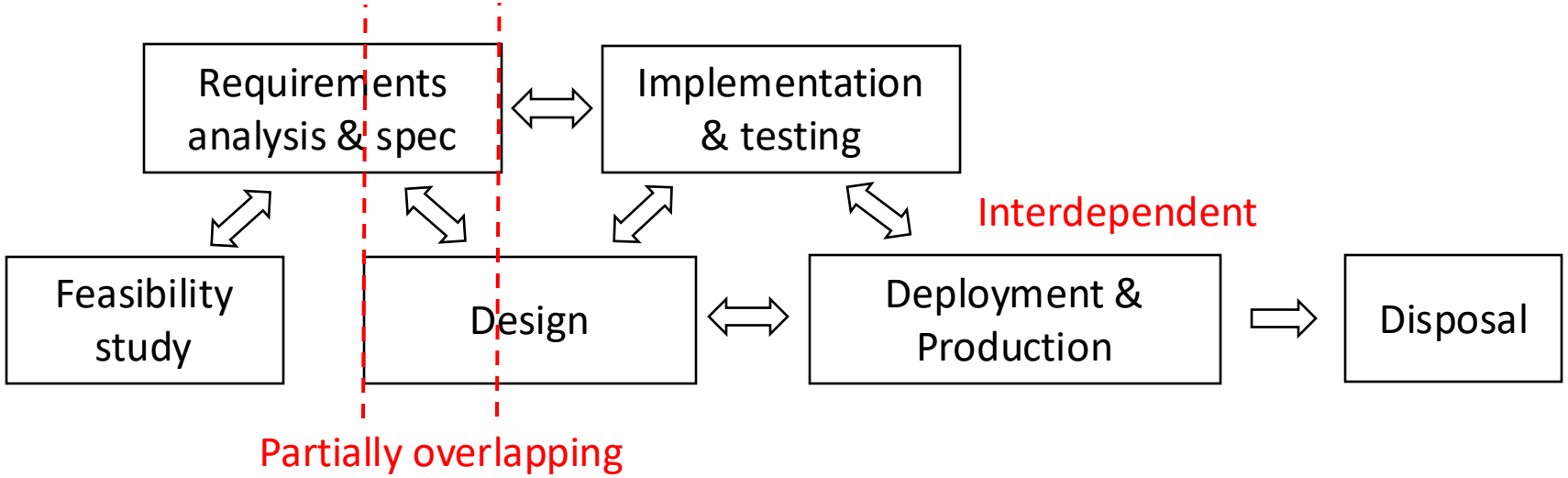
Validation:
are we making
the ”right” product?

Verification:
are we doing
the product right?

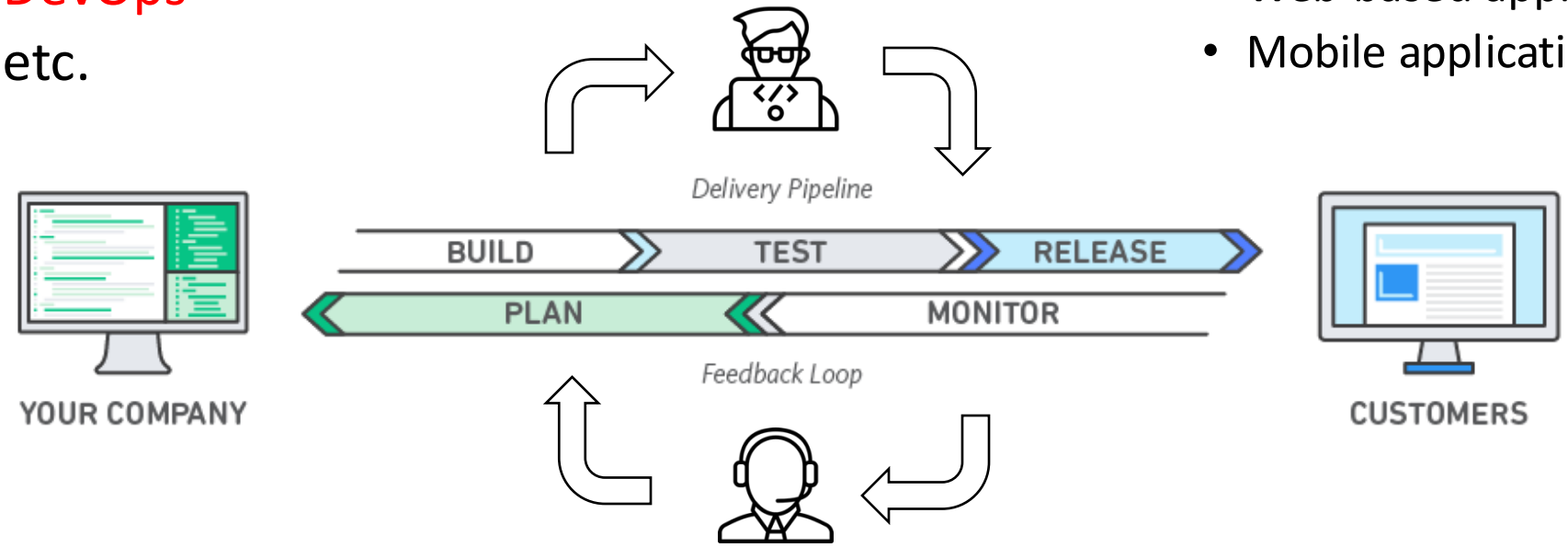
Flexible processes

- Main goal: adapt to changes (especially in requirements and specs)
- The very idea:
  - stages are not necessarily sequential
  - processes become iterative and incremental

Example



Flexible processes

- Exist in many forms
  - eXtreme Programming (XP)
  - SCRUM
  - DevOps
  - etc.

- Effective in dynamic contexts
  - Many changes per week
  - Example:
    - Web-based applications
    - Mobile applications